

Session – 1

Q-1 Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied



1. Tables & Selected Columns

- **Table 1: Restaurants**
 - restaurant_id
 - restaurant_name
- **Table 2: MenuItem**
 - restaurant_id

2. Constraint Choices & Justifications (3 Columns)

- **Restaurants.restaurant_id → PRIMARY KEY (and NOT NULL)**
 - **Justification:** It uniquely identifies each restaurant in the database.
- **Restaurants.restaurant_name → NOT NULL**
 - **Justification:** Every restaurant must have a valid name; a blank or null business name makes the listing unusable for customers.
- **MenuItem.restaurant_id → FOREIGN KEY (references Restaurants.restaurant_id)**
 - **Justification:** It establishes a relational link between a dish and the restaurant offering it.

3. Data Integrity Problems (If Constraints Are NOT Applied)

- **Entity Integrity Violation (Duplicate Records):** Without a PRIMARY KEY, multiple restaurants could share the same ID, causing ambiguous orders and lookup failures.
- **Missing/Invalid Data (Domain Integrity Issue):** Without NOT NULL on restaurant_name, entries with empty names could be inserted, breaking the UI and search functionality.
- **Orphan Records (Referential Integrity Violation):** Without a FOREIGN KEY, menu items could reference non-existent or deleted restaurant IDs, leaving "orphan" dishes in the system.

Q-2 Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?



1. Importance of the WHERE Clause in DML Statements The WHERE clause filters records so that DML operations (UPDATE, DELETE) apply only to specific rows that meet defined criteria, rather than modifying the entire dataset.

2. Application to the Given Scenarios

- **Scenario 1: Updating Stale Pending Orders (UPDATE)**

- **Role of WHERE:** Filters rows where `order_status = 'Pending'` **AND** `order_date < NOW() - INTERVAL 7 DAY`.
- **What goes wrong without WHERE:** **All orders** in the system (including newly placed, confirmed, or already delivered orders) will have their status overwritten to 'Cancelled', completely ruining transaction history.

- **Scenario 2: Deleting Unavailable Items from Closed Restaurants (DELETE)**

- **Role of WHERE:** Filters rows where `item_status = 'Unavailable'` **AND** `restaurant_status = 'Closed'`.
- **What goes wrong without WHERE:** It permanently purges the **entire menu items table**, deleting all active items across all operating restaurants and causing catastrophic data loss.

3. General Consequences of Missing/Improper WHERE Clauses

- **Irreversible Data Corruption/Loss:** Unintended mass updates or deletions cannot always be rolled back easily without transaction management or backups.
- **Loss of Business Logic & Analytics:** Accurate historical tracking (e.g., active orders vs. past revenue) is destroyed.

Q-3 Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results.



1. Required SQL Aggregate Functions

- **SUM(order_amount):** To calculate the **Total Revenue** for each restaurant.
- **AVG(order_amount):** To calculate the **Average Order Value (AOV)**.
- **COUNT(order_id):** To calculate the **Total Number of Orders** per restaurant.

2. Required SQL Clauses

- **SELECT:** To pick the restaurant identifier and the computed aggregate metrics.
- **FROM:** To specify the source table (e.g., Orders).
- **WHERE:** To filter records exclusively for the **current month before** grouping (e.g., order_date >= '2026-08-01').
- **GROUP BY:** To group individual order records by each restaurant (GROUP BY restaurant_id).
- **HAVING:** To filter grouped results where the order count exceeds 20 (HAVING COUNT(order_id) > 20).

3. Why HAVING Must Be Used Instead of WHERE

- **Execution Order / Pipeline:** The WHERE clause filters individual rows **before** any aggregation or grouping happens. At the time WHERE executes, the database does not know the total count of orders for a restaurant.
- **Aggregated Conditions:** COUNT(order_id) > 20 is a condition based on a summarized/aggregated result across multiple rows.
- **Role of HAVING:** The HAVING clause runs **after** the GROUP BY operation, allowing the query engine to filter groups based on aggregate function values. Writing WHERE COUNT(order_id) > 20 would result in a SQL syntax error.

Q-4 Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table.



1. Required SQL Aggregate Functions

- **SUM(order_amount):** To calculate the **Total Revenue** for each restaurant.
- **AVG(order_amount):** To calculate the **Average Order Value (AOV)**.
- **COUNT(order_id):** To calculate the **Total Number of Orders** per restaurant.

2. Required SQL Clauses

- **SELECT:** To pick the restaurant identifier and the computed aggregate metrics.
- **FROM:** To specify the source table (e.g., Orders).
- **WHERE:** To filter records exclusively for the **current month before** grouping (e.g., order_date >= '2026-08-01').
- **GROUP BY:** To group individual order records by each restaurant (GROUP BY restaurant_id).
- **HAVING:** To filter grouped results where the order count exceeds 20 (HAVING COUNT(order_id) > 20).

3. Why HAVING Must Be Used Instead of WHERE

- **Execution Order / Pipeline:** The WHERE clause filters individual rows **before** any aggregation or grouping happens. At the time WHERE executes, the database does not know the total count of orders for a restaurant.
- **Aggregated Conditions:** COUNT(order_id) > 20 is a condition based on a summarized/aggregated result across multiple rows.
- **Role of HAVING:** The HAVING clause runs **after** the GROUP BY operation, allowing the query engine to filter groups based on aggregate function values. Writing WHERE COUNT(order_id) > 20 would result in a SQL syntax error.

Q-5 Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?



1. Justifications for Using a VIEW Over Direct Table Access

- **Data Security & Column Masking:** A VIEW restricts access to sensitive or internal information by hiding raw database keys (restaurant_id, order_id) and exposing only the necessary aggregated columns to the front-end team.
- **Simplified Query Complexity (Abstraction):** The front-end team doesn't need to write complex SQL joins, GROUP BY, or aggregate calculations (SUM, AVG, COUNT). They can query top_restaurants like a standard table with a simple SELECT * FROM top_restaurants.
- **Consistency & Centralized Logic:** Business metrics (e.g., how "top" performance and "average rating" are calculated) are maintained centrally in the database definition, preventing inconsistent formulas across multiple front-end pages.

2. Two Limitations the Front-End Developer Should Be Aware Of

- **Read-Only / Non-Updatable:** Because this VIEW uses aggregate functions (COUNT(orders), AVG(rating)) and GROUP BY, it is non-updatable. The front-end cannot perform INSERT, UPDATE, or DELETE operations through this VIEW.
- **Performance Overhead (Compute on Query):** Standard views do not store physical data; the underlying joins and aggregations execute each time the front-end queries the VIEW. Without indexing or materialized caching, frequent queries on large datasets can lead to slow dashboard load times.

Q-6 Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.



1. Required Database Mechanism

- **Database Transactions / ACID Properties (Specifically Atomicity):**
 - **Atomicity ("All or Nothing"):** Ensures that multiple related database operations are treated as a single indivisible unit of work. Either all operations execute successfully, or the entire transaction is cancelled.

2. How Transaction Commands Work Together

- **START TRANSACTION / BEGIN:**
 - Initiates the payment workflow, bundling the wallet deduction and payment record insertion into one logical unit.
- **COMMIT:**
 - If **both** the wallet deduction and the payment record insertion succeed without errors, COMMIT permanently saves all changes to the database.
- **ROLLBACK:**
 - If the payment record insert fails (or an exception occurs at any point), ROLLBACK undoes the wallet deduction, restoring the customer's balance back to its original state before the transaction started.
- **SAVEPOINT:**
 - Sets intermediate checkpoints within a complex transaction (e.g., SAVEPOINT after_wallet_deduct;). It allows rolling back specific partial operations without aborting the entire transaction if an optional step fails.